

포트폴리오

수석 시스템 소프트웨어 엔지니어 & AI 스페셜리스트

"20 년의 시스템 엔지니어링 견고함 위에 AI 의 혁신을 더하다"

Email c0z0c@outlook.com

GitHub [c0z0c](https://github.com/c0z0c)

PyPI [Projects](#)

Patents [10 Registered](#)

소개

"Mission-Critical 환경의 신뢰성을 AI 서비스에 이식하는 엔지니어"

철도 및 전력 시스템 분야에서 20 년간 종사하며, 고신뢰성 시스템을 개발해왔습니다. IEC 62443 보안 표준과 KTC 인증을 준수하는 소프트웨어를 설계하며, 10 건의 특허 등록을 통해 기술적 난제 해결 능력을 입증했습니다.

현재는 이러한 엔지니어링 베이스 위에 LLM, Vision, RAG 기술을 접목하여, 단순한 모델 구현을 넘어 '서비스 가능한(Serving) 견고한 AI 시스템'을 구축하는 데 주력하고 있습니다.

기술 스택

구분 보유 기술

AI & ML	PyTorch	LangChain	OpenAI	FAISS
Backend	FastAPI	Flask	Docker	Google Cloud
System	C++	Linux	Qt	Embedded

주요 프로젝트

AI 엔지니어링

1. [팀] 나노코코아 - SaaS 광고 콘텐츠 생성 플랫폼

Role: 시스템 아키텍트 & 데이터 파이프라인 | Tech: FastAPI, MSA, Docker, FLUX.1-dev

* 개요: 소상공인을 위한 광고 문구·이미지·랜딩페이지 원스톱 자동 생성 시스템

* 핵심 성과:

* 속도 혁신: 초기 Load/Unload 방식(600 초)의 한계를 경량 모델 상주(In-Memory)

아키텍처로 고도화하여 90% 단축 (60 초)

* 자원 최적화: L4 GPU(24GB) VRAM 제약을 극복하기 위해 최적 경량 모델 발굴 및 FP16

양자화 적용

* 문제 해결: Diffusion 모델의 한글 렌더링 한계를 HTML/CSS 텍스트 렌더링 후 합성

방식으로 대체하여 오타율 0% 달성

* GitHub 저장소 : <https://github.com/c0z0c/codeit-ai-3team-ad-content>

* 시연 영상 : <https://youtu.be/YG-xEI0PDD8>

2. [팀] RAG 기반 나라장터 입찰 문서 검색 시스템

Role: 임베딩 엔지니어 (PM) | Tech: LangChain, FAISS, OpenAI, Streamlit

* 개요: 공공조달 PDF/HWP 문서 내 복잡한 정보를 자연어 질의로 추출하는 RAG 시스템

* 핵심 성과:

* 비용 절감: 전처리 최적화 및 불필요한 메타데이터 제거로 원본 대비 토큰 절감

* 검색 품질: 파일 해시(SHA-256) 기반 중복 제거 로직 구현 및 HWP 표 데이터 마크다운 변환 파이프라인 자체 개발

* GitHub 저장소 : https://github.com/c0z0c/codeit_ai_g2b_search

* 시연 영상: <https://youtu.be/rFoyNxsR5cE>

3. [개인] H2 Map - 실시간 수소충전소 지도 서비스

Tech: Google Cloud Run, Flask, Kakao Map API

* Serverless 아키텍처: Cloud Run 을 활용하여 트래픽이 없을 때 비용이 발생하지 않는 비용 효율적 구조(Scale-to-Zero) 구현

* 실시간 처리: 공공데이터 포털 API 연동 및 WebSocket 알람 시스템 구축

* 서비스 바로가기 : <https://h2map-p4wp5hxaa-du.a.run.app>

4. [개인] PDF RAG 검색 시스템

Tech: OpenAI Embeddings, FAISS, pymupdf4llm

* 2-Stage 검색: '요약문 검색 → 원본 정밀 검색' 파이프라인으로 Hallucination 최소화 및 정확도 향상

* GitHub 저장소 : https://github.com/c0z0c/pdf_search_openai

* 시연 영상 : <https://youtu.be/Iv18gd7ouDA>

오픈소스 기여

개발 생산성을 높이는 도구를 직접 제작하여 PyPI 에 배포하고 유지보수합니다.

- helper-dev-utils: 로깅, 디버깅 등 파이썬 개발 생산성 도구
- helper-plot-hangul: OS 별(Windows/Linux/Mac) 폰트 자동 탐지 및 Matplotlib 한글 깨짐 해결
- helper-md-doc: 파이썬 객체(Docstring)를 분석하여 Markdown 문서를 자동 생성
- helper-hwp: RAG 전처리를 위한 HWP 텍스트 추출 및 마크다운 변환 라이브러리

시스템 엔지니어링 경력 (Domain Expertise)

철도 통합 방송 및 한전 시스템 (2011 - 2025)

임베디드 리눅스 및 RTOS 환경에서의 고성능 시스템 개발 및 표준화 수행

- 가상 시뮬레이션 환경 구축 (Digital Twin):
 - 실제 차량 테스트의 물리적/시간적 제약을 극복하기 위해 차량 신호 시뮬레이터 독자 개발
 - 개발 및 디버깅 효율 향상 및 현장 리스크 최소화 (관련 특허 등록)
- Mission-Critical 보안 시스템 설계:
 - 백신 설치가 불가능한 폐쇄망 제약을 극복하기 위해 경량 해시체인 무결성 검증 시스템 설계
 - 국제 보안 표준 IEC 62443-4-1 기반 보안 설계 및 KCMVP 암호화 모듈 적용

특허 및 교육

보유 특허 (총 10 건)

문제 해결을 위해 고안한 핵심 기술들을 특허로 권리화하였습니다.

- * JSONL 포맷 압축 및 복원 방법 (AI 데이터 처리 관련, 출원 중)
- * 열차방송 시뮬레이션 장치 (등록)
- * PLC 네트워크 신호 간섭 판단 시스템 (등록)

교육

- * Codeit AI 엔지니어 트랙 4기 수료 (2025.07 - 2026.01)

Pretendard

지향성 마이크 성능 시연용 Realtime AI 키오스크 솔루션

초저지연 음성 대화 시스템

Senior Software Engineer | 2026. 02

  FastAPI  

Vapi.ai / OpenAI



미션: 소음 속에서도 즉각 반응하는 데모 환경 구축



The Goal

고성능 지향성 마이크의 소음 차단(Noise Canceling) 및 수음 능력을 직관적으로 시연.

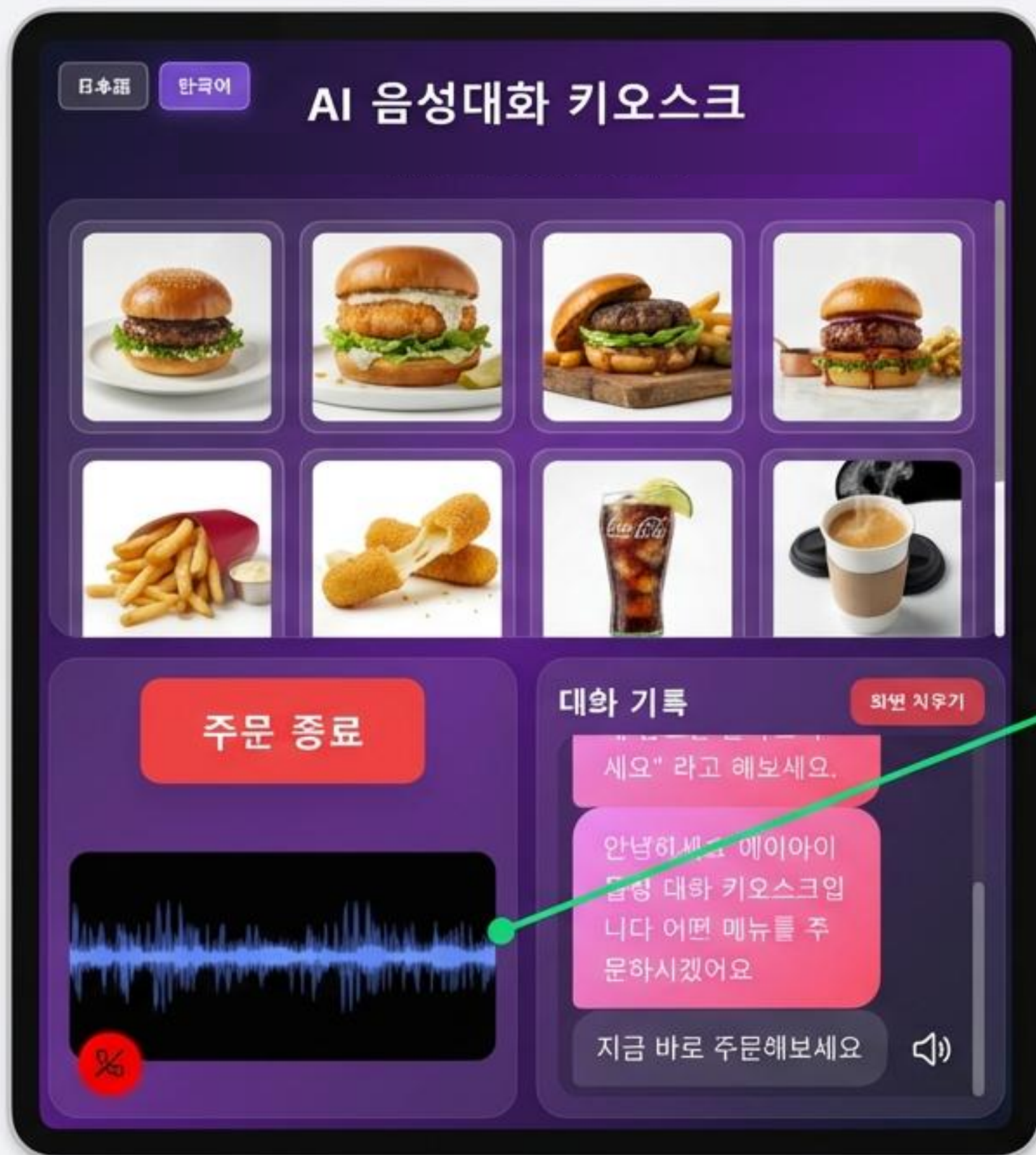
The Constraint

현장 소음 환경에서도 AI가 사용자의 의도를 정확히 파악해야 함.



Key Objectives

- **High-Responsiveness:** '자연스러운 대화 속도'를 위한 지연시간(Latency) 최소화.
- **Zero-Touch UI:** 버튼 조작 없이 목소리만으로 주문부터 결제까지 완결.
- **Make Hardware Shine:** 소프트웨어적 기교보다는 하드웨어의 입력 성능을 투명하게 보여주는 역할.



보이지 않는 인터페이스 (Invisible Interface)

- **Hands-Free Interaction**

고성능 VAD(Voice Activity Detection)를 적용하여 버튼 클릭 없는 자연스러운 발화 감지.

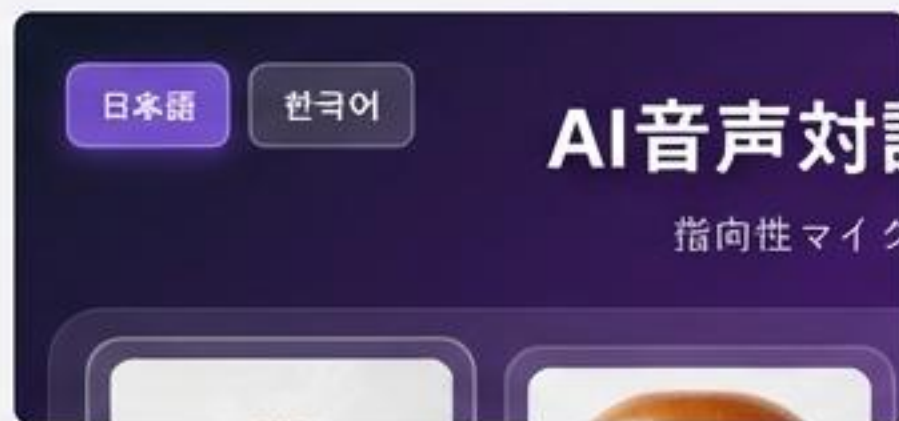
- **Real-time Visualizer**

마이크 입력 신호를 실시간 파형으로 시각화하여 하드웨어 동작 여부를 직관적으로 피드백.

- **Live Chat UI**

음성 인식 결과(STT)와 AI 응답을 채팅 형태로 보여주어 인식 정확도를 즉시 검증.

초저지연 AI 대화 엔진 (Core Engine)



Multilingual Support

한국어/일본어 실시간 전환 및
각 언어별 전용
페르소나(Assistant ID) 매핑.

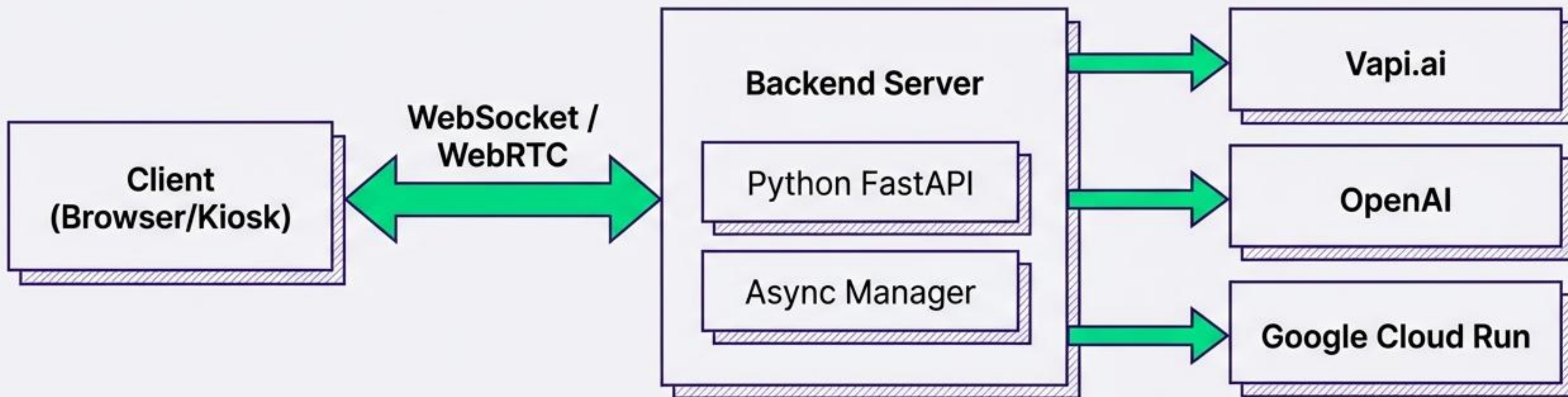
Scenario Logic

주문 → 확인 → 가상 결제 →
호출로 이어지는 완결된
프랜차이즈 시나리오 구현.

Fallback Strategy

Vapi.ai 미들웨어와 OpenAI
직접 연결 방식을 상황에 따라
선택 가능한 유연한 설계.

견고한 백엔드 아키텍처 (System Architecture)



- **Server Framework:** Python FastAPI (Async 비동기 처리 지원).
- **Communication:** WebSocket을 이용한 양방향 실시간 데이터 스트리밍.
- **Data Flow:** 오디오 스트림과 제어 신호(Control Signals)의 효율적인 라우팅.
- **Dependency Injection:** 기기 인증 및 DB 연결을 위한 모듈화된 의존성 주입 패턴 사용.

중앙 집중식 환경 변수 관리 시스템

Problem: 데모 현장에서 API 키나 프롬프트 변경 시 서버 재배포 불가능.

Solution: Firestore 기반의 실시간 Config 관리 대시보드 구축.

- **Hot-Swapping:** 서버 재시작 없이 API Key, Assistant ID, Test Mode 등 핵심 설정 즉시 변경.
- **Security:** SHA-256 해시 기반의 관리자 비밀번호 인증.
- **Visual Editor:** JSON 형식이 아닌 직관적인 Web UI 제공.

환경 변수 설정

▶ 관리자 비밀번호 변경

AMPLIFICATION	4
AUTH_UUID_ENABLE	false
HOST	0.0.0.0
MENU_INFO_ENABLED	false
PORT	8080
RECOGNITION_TEXT_ENABLED	false
TEST_MODE	false
USE_FIRESTORE	true
VAPI_ASSISTANT_JA_ID	a248b6a0-47ad-43b1-9c4d-2976a5b50de4
VAPI_ASSISTANT_KR_ID	d1ddfc8f-4f26-4490-94fa-ccdf9bab8f1c
VAPI_KEY	5179a2d3-aa8a-48d6-88bc-85bb43a1d1e0

저장

기기 인증 및 접근 제어 (Security & Access Control)

UUID Whitelist

브라우저 로컬 스토리지에 생성된 UUID를 기반으로 승인된 기기만 API 호출 허용.

Expiration Control

만료일(Expires_at) 설정을 통해 임시 데모용 기기의 접근 권한 자동 회수.

기기 인증 관리						
인증 상태: X AUTH_UUID_ENABLE 비활성화됨 (모든 기기 접근 허용)						
기기 ID	상태	생성일시	최근 접속	만료 일	메모	작업
339d4b17-df1a-4fdf-b70e-8af547fadbd7	APPROVED	2026. 02. 07. 오후 12:47	2026. 02. 07. 오후 01:35	-	명원개발 PC(google)	수정 삭제
bc7319b0-966b-427d-b137-2d45d6c81b89	APPROVED	2026. 02. 06. 오후 03:58	2026. 02. 07. 오후 12:20	-	명원개발 PC(local)	수정 삭제
c80e7a1b-96aa-4d2a-a00f-7fe016172586	APPROVED	2026. 02. 07. 오후 01:17	2026. 02. 07. 오후 01:43	-	명원개발 PC(local)	수정 삭제

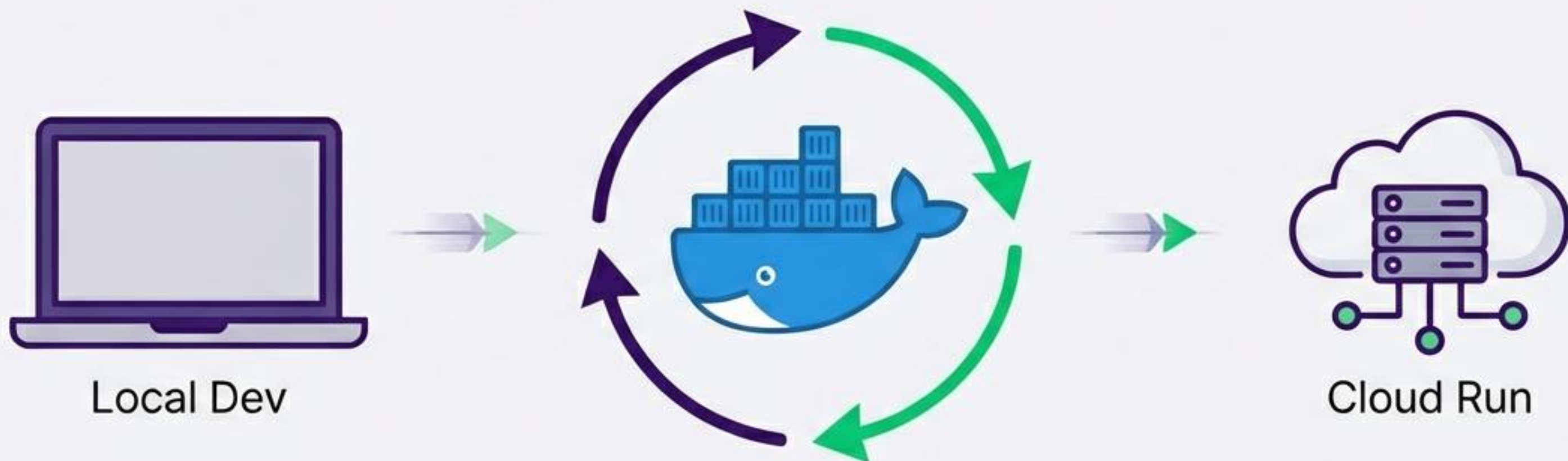
Lifecycle Management

승인(Approved), 대기(Waiting), 거부(Rejected) 상태 관리.

Cost Management

무분별한 토큰 사용 방지를 위한 엄격한 세션 제어.

어디서든 실행 가능한 배포 환경 (DevOps)



Containerization

Dockerfile을 통한 환경 표준화
(개발, 테스트, 운영 환경 일치).



Cloud & Local

Google Cloud Run을 통한 서버리스
배포 및 로컬 노트북 단독 구동 모두 지원.



Stability

WebRTC 및 WebSocket 연결
유지를 위한 네트워크 최적화 설정.



프로젝트 성과 및 핵심 가치

- ✓ **Performance**
하드웨어(마이크) 성능 검증을 위한 신뢰성 높은 테스트 베드 역할 수행.
- ✓ **Completeness**
음성 인식부터 관리자 대시보드까지 Full-Cycle 자체 개발.
- ✓ **Reliability**
다국어(KR/JP) 환경에서의 안정적인 WebSocket 연결 및 예외 처리.
- ✓ **Documented**
Swagger/ReDoc을 통한 API 문서화 및 상세 개발 보고서 작성.



Thank You

김명환 (Senior Software Engineer)

c0z0c@naver.com

GitHub / Portfolio Link



familydoctor포트폴리오

1. 프로젝트 개요

항목	내용
프로젝트명	FamilyDoctor - AI 주치의
유형	개인 프로젝트
기간	2026년
배포 URL	https://familydoctor-sc6ue6cf3q-uc.a.run.app
저장소	https://github.com/c0z0c/familydoctor

가족 구성원의 건강 히스토리를 누적 관리하고, Gemini LLM이 오케스트레이터로 대화 흐름을 조절하며 MedGemma 4B GGUF 모델이 로컬에서 직접 의료 조언을 생성하는 웹 애플리케이션.

2. 기술적 도전 과제와 해결

2.1 로컬 LLM(GGUF)과 클라우드 서비스의 통합

문제: Google Cloud Run은 stateless 컨테이너 환경으로 2.5GB GGUF 모델을 매번 로드해야 한다. 모델 파일을 이미지에 포함하면 빌드/배포 시간이 급증하고, 환경 변수나 DB 연결처럼 일반적인 "준비 완료" 판단이 불가능하다.

해결:

GCS 버킷 → GCS FUSE → /mnt/models (Cloud Run gen2 볼륨 마운트)
└─ 이미지 크기 유지 + 모델 파일 분리 관리

- MedGemmaService를 별도 자식 프로세스(multiprocessing.spawn)로 분리하여 FastAPI 메인 프로세스와 메모리 공간 격리
- 부모-자식 간 통신을 multiprocessing.Queue로 처리, 추론 요청/응답을 비동기로 위임
- prepare_async()를 asyncio.create_task()로 백그라운드 실행하여 서버는 즉시 기동

2.2 Cloud Run cold start에서의 503 문제

문제: Cloud Run 기본 TCP probe는 uvicorn 포트가 열리면 즉시 Ready로 판단한다. GGUF 모델 예열(~41초)이 완료되기 전에 트래픽이 유입되어 /api/medical/chat이 503을 반환하는 상황이 반복됐다.

실측 데이터 (Cloud Run 로그, revision familydoctor-00014-wg7):

단계	시각	경과
워커 프로세스 기동 + 예열 대기 시작	08:22:39	0초
llama_context 출력 (모델 파일 읽기 완료)	08:23:16	+36.7초 (90%)
예열 추론 완료 (16토큰)	08:23:20	+40.6초 합계
/health 200 (startup probe 통과)	08:23:20	워커 완료와 동시
Cloud Run revision Ready	08:23:20	컨테이너 시작부터 1분 12초

해결:

1. /health 엔드포인트가 model_ready=False일 때 HTTP 503 반환

2. cloudbuild.yaml에 startup probe 추가

```
--startup-probe=httpGet.path=/health,httpGet.port=8080,  
initialDelaySeconds=10,periodSeconds=10,  
failureThreshold=90,timeoutSeconds=5
```

3. startup probe가 /health 200을 확인한 후에만 트래픽 전환 → 503 완전 차단

2.3 SIGTERM 경합 조건 버그

문제: Cloud Run이 인스턴스 교체 시 SIGTERM을 보내면 `_on_sigterm` 핸들러가 `model_ready=False`로 설정한다. 그러나 백그라운드 예열 Task가 이후에 완료되면서 `model_ready=True`로 되돌려 SIGTERM을 받은 인스턴스가 다시 트래픽을 받는 문제가 발생했다.

해결:

```
# MedGemmaService  
self._sigterm_received: bool = False  
  
# main.py _on_sigterm 핸들러  
medgemma._sigterm_received = True  
medgemma.model_ready = False  
  
# prepare_async() 완료 직전 가드  
if self._sigterm_received:  
    logger.warning("SIGTERM 수신 상태 - model_ready=True 설정 건너뛴")  
    return  
self.model_ready = True
```

2.4 LangGraph ReAct Agent Tool 설계

문제: 단순 Gemini API 직접 호출(GeminiService 레거시)로는 대화 흐름 제어, 로그 저장, MedGemma 호출을 하나의 요청 안에서 일관성 있게 처리하기 어려웠다.

해결: LangGraph `create_react_agent`로 전환하여 5개 Tool을 클로저 바인딩:

Tool	역할
<code>clarify_user</code>	PQRST 문진 프로토콜 기반 추가 질문
<code>check_outcome</code>	미완료 로그 경과 확인
<code>consult_medgemma</code>	MedGemmaService 로컬 추론 호출
<code>save_log</code>	증상 로그 Firestore 저장
<code>complete_log</code>	미완료 로그 완료 처리

Gemini LLM(`gemini-3.1-flash-lite-preview`)이 PQRST 문진 가이드와 중복 질문 금지 규칙이 포함된 시스템 프롬프트를 기반으로 Tool을 자율 선택한다.

2.5 비용 최소화 설계

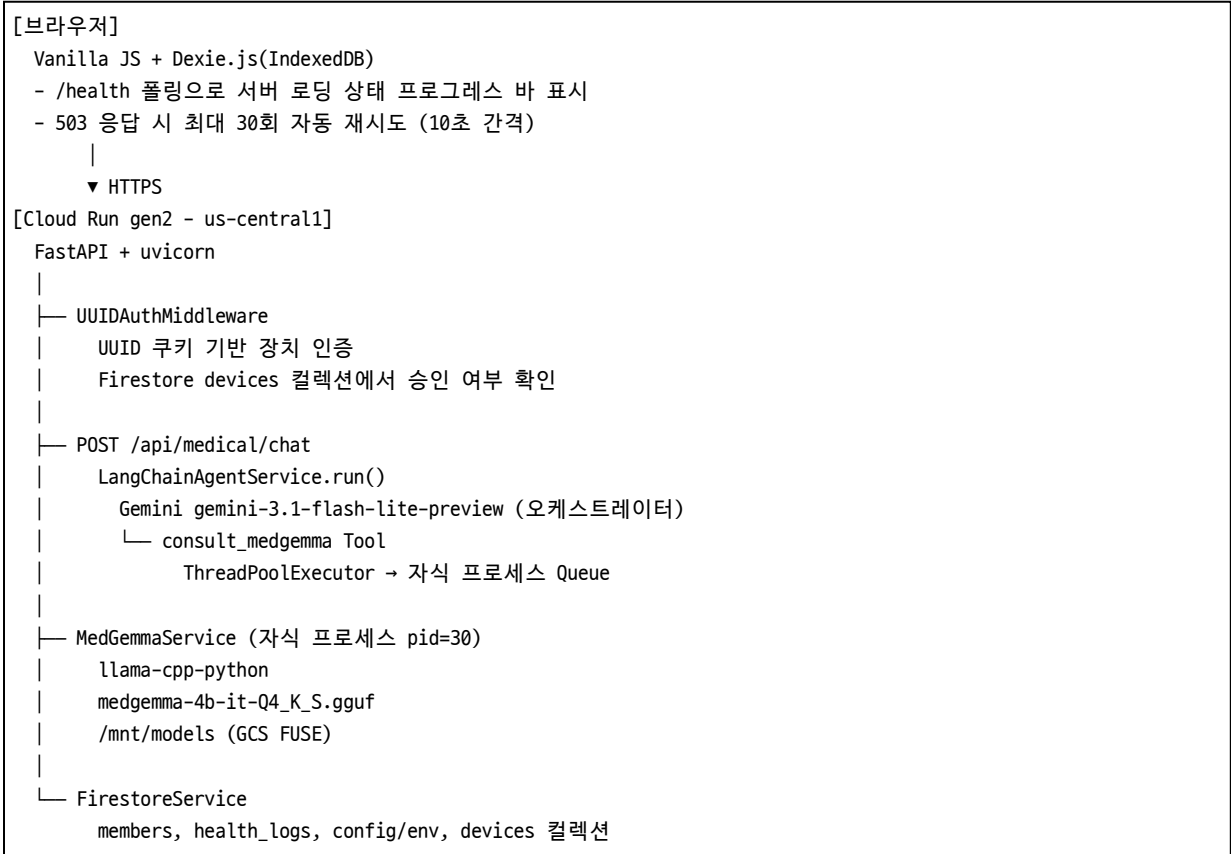
월 1~2회 호출하는 개인 프로젝트 특성상 비용 최소화가 핵심 요구사항이었다.

설정	이전	이후	이유
<code>min-instances</code>	0	0	idle 비용 없음 유지
<code>max-instances</code>	3	1	동시 사용자 없음

concurrency	10	1	단일 사용자
cpu-boost	활성	비활성	cold start 중 CPU 2배 과금 제거

예상 월 비용:
0.10 미만 (GCS 스토리지 0.05 포함)

3. 시스템 아키텍처



4. 핵심 구현 코드

4.1 MedGemmaService 멀티프로세스 구조

```

# 자식 프로세스에서 모델 1회 로드 후 Queue로 요청 수신
def _worker_loop(req_queue, res_queue, model_path):
    llm = Llama(model_path=model_path, n_ctx=4096, n_threads=1)
    while True:
        req = req_queue.get()
        if req is None: # 종료 신호
            break
        raw = llm.create_chat_completion(messages=req["messages"], ...)
        res_queue.put({"ok": True, "result": raw["choices"][0]["message"]["content"]})

# 부모 프로세스(FastAPI)에서 비동기 위임
async def prepare_async(self):
    await loop.run_in_executor(self._executor, self._ensure_worker)
    await loop.run_in_executor(self._executor, self._warmup) # 실측 40.6초
    if not self._sigterm_received:
        self.model_ready = True

```

4.2 /health startup probe 연동

```

@app.get("/health")
async def health_check(request: Request):
    medgemma = request.app.state.medgemma
    body = {
        "status": "healthy" if medgemma.model_ready else "initializing",
        "model_ready": medgemma.model_ready,
        "model_downloading": medgemma.model_downloading,
    }
    if not medgemma.model_ready:
        return JSONResponse(content=body, status_code=503)
    return body

```

4.3 프론트엔드 서버 로딩 폴링

```

async function waitForServer() {
    if (await checkServerReady()) return; // 이미 warm이면 즉시 통과

    overlay.classList.remove("hidden"); // 프로그레스 바 오버레이 표시
    // 1초마다 경과 시간 갱신, 10초마다 /health 폴링
    // model_ready=true 되면 오버레이 자동 제거
}

```

5. 개발 회고

잘 된 점

- GCS FUSE 볼륨 마운트로 이미지 크기와 모델 파일 관리 분리 성공
- startup probe 도입으로 cold start 중 503 문제 완전 해결

- SIGTERM 경합 조건을 _sigterm_received 플래그로 명확하게 가드
- LangGraph ReAct Agent 전환으로 대화 흐름 제어 코드가 선언적으로 단순화

개선 여지

- GCS FUSE 네트워크 읽기로 인한 cold start 5~6분은 모델 파일을 Docker 이미지에 포함하면 30~60초로 단축 가능 (Artifact Registry 비용 증가 트레이드오프)
- n_ctx=4096으로 제한 중 (n_ctx_train=131072). 메모리 확보 시 확장 가능
- n_threads=1로 단일 스레드 추론 중. CPU 2개 활용을 위해 조정 가능
- 개인 프로젝트로 비용은 최소화하였으나 더 빠른 응답이 필요하면 항상 켜진 인스턴스(min-instances=1)로 변경 고려 가능

6. 기술 스택 요약

Language	: Python 3.12
Framework	: FastAPI, LangChain, LangGraph
LLM	: Google Gemini gemini-3.1-flash-lite-preview (오케스트레이터) MedGemma 4B GGUF Q4_K_S (로컬 추론, llama-cpp-python)
Database	: Google Cloud Firestore
Frontend	: Vanilla JS, Dexie.js, marked.js
Infra	: Google Cloud Run gen2, GCS FUSE, Cloud Build, Artifact Registry
Auth	: UUID 기반 장치 인증 (Firestore devices 컬렉션)

포트폴리오: 텔레그램 + n8n + Notion 기반 근태관리 자동화 시스템

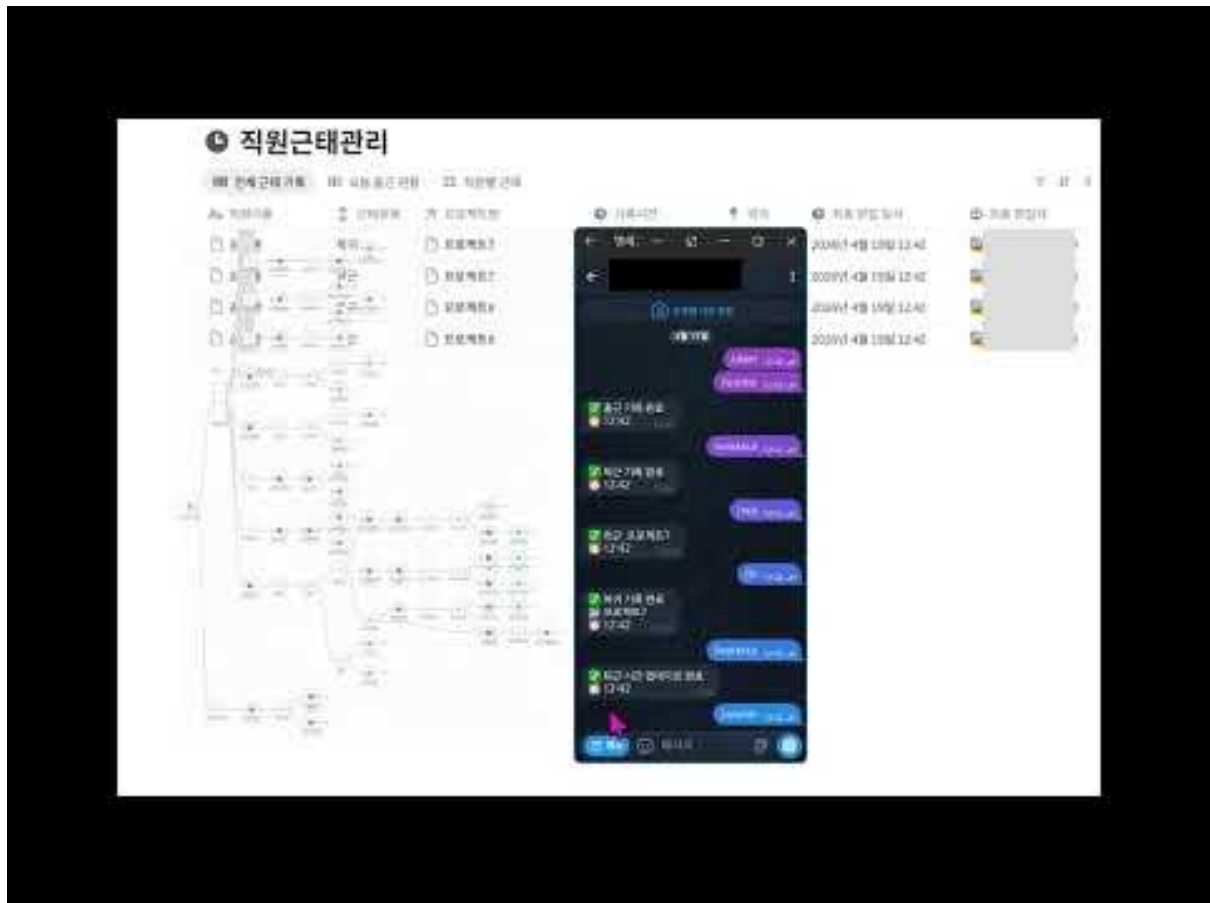
프로젝트 개요

텔레그램 봇 명령어로 직원 근태를 기록하고, 이를 Notion 데이터베이스에 자동 저장·조회·수정할 수 있도록 구축한 업무 자동화 시스템이다.

단순한 봇 연동을 넘어서, self-hosted n8n 환경과 외부 webhook 수신 구조, 데이터 중심 운영 설계, 예외 처리 흐름까지 포함한 실운영형 자동화 시스템으로 설계했다.

데모 영상

- 데모 영상: <https://youtu.be/eV88hV7wLAq>



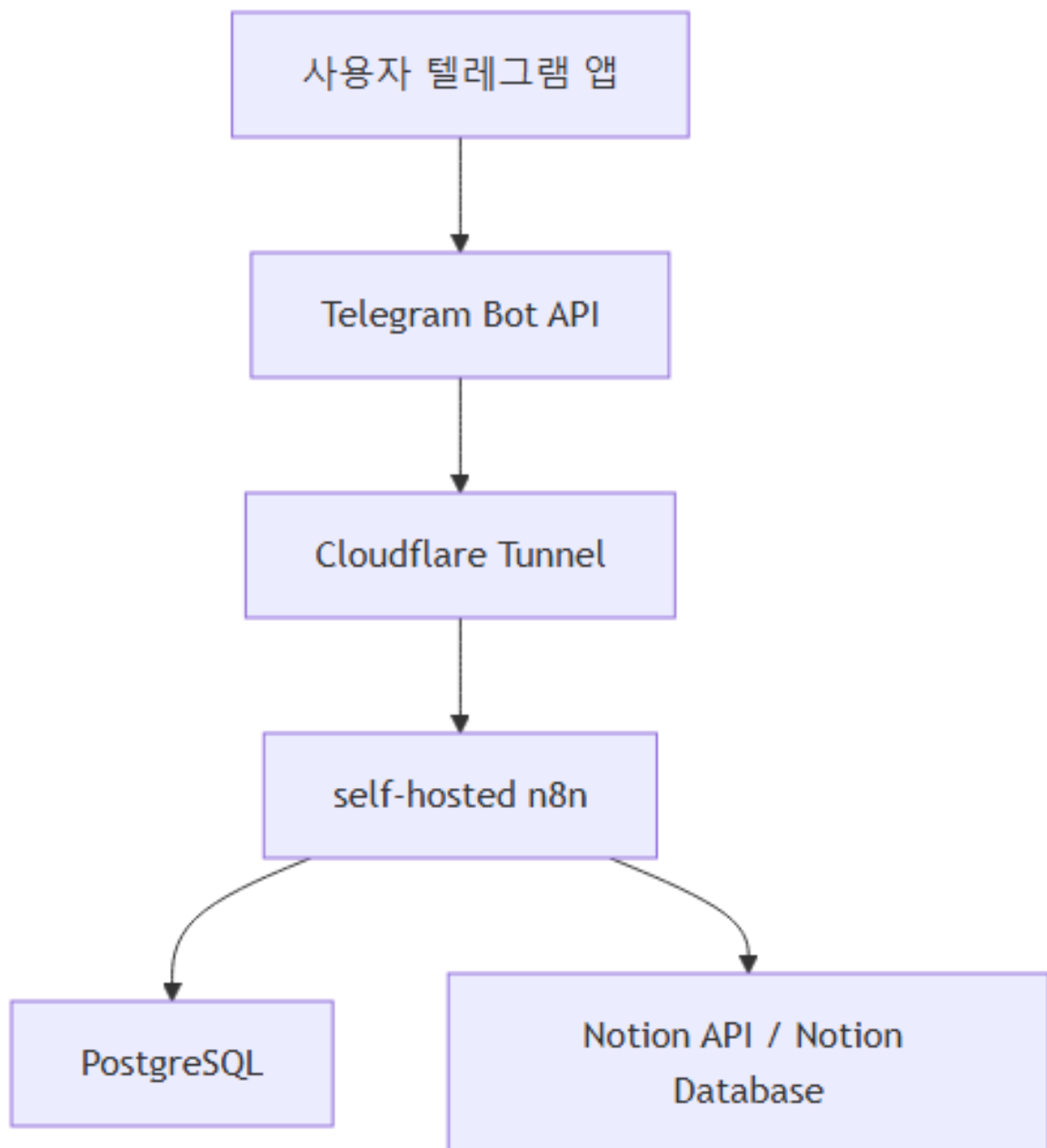
이 프로젝트의 목표는 다음과 같았다.

- 반복적인 근태 입력 과정을 메신저 명령어 기반으로 단순화
- 운영자가 코드 수정 없이 명령어와 설정을 변경할 수 있도록 데이터 중심 구조 설계
- 외부 접속 제약, 한글 컬럼 처리, 플랫폼 제한사항 등 실제 운영 환경의 문제를 안정적으로 해결

핵심 성과

- 텔레그램 명령어만으로 출근, 퇴근, 외근, 복귀, 조회, 삭제, 수정까지 처리하는 자동화 흐름 구현
 - 명령어, 사용자 설정, 프로젝트 정보를 데이터베이스로 분리해 운영 중 무코드 변경 가능 구조 설계
 - 외부 webhook 연결 제약이 있는 환경에서도 안정적으로 이벤트를 수신하는 self-hosted 자동화 환경 구성
 - n8n 기본 노드 한계를 JavaScript Code 노드로 보완해 실사용 가능한 사용자 경험 구현
 - 다수 사용자 알림 발송을 병렬 처리해 순차 방식 대비 처리 성능 개선
-

시스템 구성



구성 요소

구성 요소	역할
n8n	자동화 워크플로우 실행 엔진
PostgreSQL	워크플로우 및 설정 영속 저장
Cloudflare Tunnel	외부 webhook 수신 경로 안정화
Notion	근태, 명령어, 프로젝트 데이터 저장소
Telegram Bot	사용자 입력 및 결과 확인 인터페이스

주요 기능

1. 근태 기록 자동화

텔레그램 명령어를 통해 출근, 퇴근, 외근, 복귀를 기록할 수 있도록 구성했다.

- 당일 기록 조회
- 기록 삭제
- 프로젝트 지정 외근 등록
- 기존 기록의 프로젝트 정보 수정
- 상황에 따라 중복 허용, 갱신, 차단 정책이 다른 명령어 구조 적용

2. 데이터 중심 명령어 관리

명령어 정의를 코드에 하드코딩하지 않고 별도 데이터베이스로 분리했다.

이를 통해 운영자는 데이터만 수정해도 새 명령어를 추가하거나 설명을 바꿀 수 있고, 봇 메뉴도 자동 동기화되도록 구성했다.

3. 사용자별 근태 분리 처리

조회와 수정 흐름 전반에 사용자 식별 기준을 일관되게 적용해, 사용자별 기록이 섞이지 않도록 설계했다.

또한 알림 발송에 필요한 사용자별 설정 정보를 분리 관리해, 개인별 출근 기준 시각에 맞는 안내가 가능하도록 구성했다.

4. 외근/복귀 흐름 자동화

복귀 입력 시 당일 외근 기록 중 아직 짝이 맞지 않은 최근 항목을 우선 탐지해 자동 연결하도록 구현했다.

자동 탐지에 실패하면 수동 선택 흐름으로 자연스럽게 전환되도록 설계해, 자동화와 안정성을 함께 확보했다.

기술적으로 해결한 문제

1. 외부 webhook 수신 제약 해결

일반적인 공개형 환경이 아닌 self-hosted 환경에서 외부 메신저 플랫폼의 webhook을 안정적으로 수신해야 했다.

이를 위해 Cloudflare Tunnel 기반 연결 구조를 도입해, 직접 공개가 어려운 환경에서도 안정적으로 이벤트를 수신할 수 있도록 구성했다.

2. 한글 컬럼 처리 문제 해결

n8n의 Notion 연동 과정에서 한글 속성명이 정상적으로 매핑되지 않는 문제가 있었다. 이 문제는 Notion 노드의 단순화 옵션을 끄고 원본 응답을 유지하는 설정으로 해결했다. 즉, 코드로 우회한 것이 아니라 플랫폼이 제공하는 올바른 옵션을 적용해 데이터 손실을 막은 사례에 가깝다. 이후 워크플로우에서는 필요한 필드만 읽도록 최소한의 후처리만 적용해 안정성을 유지했다.

3. 날짜 조건 필터 한계 보완

플랫폼 기본 필터만으로는 복수 날짜 조건을 안정적으로 조합하기 어려운 구간이 있었다. 이를 보완하기 위해 조회 범위를 단순화하고, 실제 업무 규칙에 필요한 상세 필터링은 Code 노드에서 수행하도록 역할을 분리했다.

4. Telegram callback 데이터 길이 제한 대응

인라인 버튼 기반 수정 기능을 구현하는 과정에서 플랫폼의 callback_data 길이 제한을 고려해야 했다. 식별자를 압축해 전달하고, 서버 측에서 다시 복원하는 방식으로 사용자 경험을 유지하면서 제약을 해결했다. 예시 포맷은 `ep|{recordIdCompressed}|{projectIdPrefix}` 형태로 설계했다.

5. Code 노드 문자열 처리 안정화

워크플로우 JSON 내부에 포함되는 JavaScript 코드 특성상 문자열 표현 방식에 따라 저장 오류가 발생할 수 있었다. 이 문제는 템플릿 문자열 사용을 줄이고, 더 안전한 문자열 결합 방식으로 변경해 해결했다.

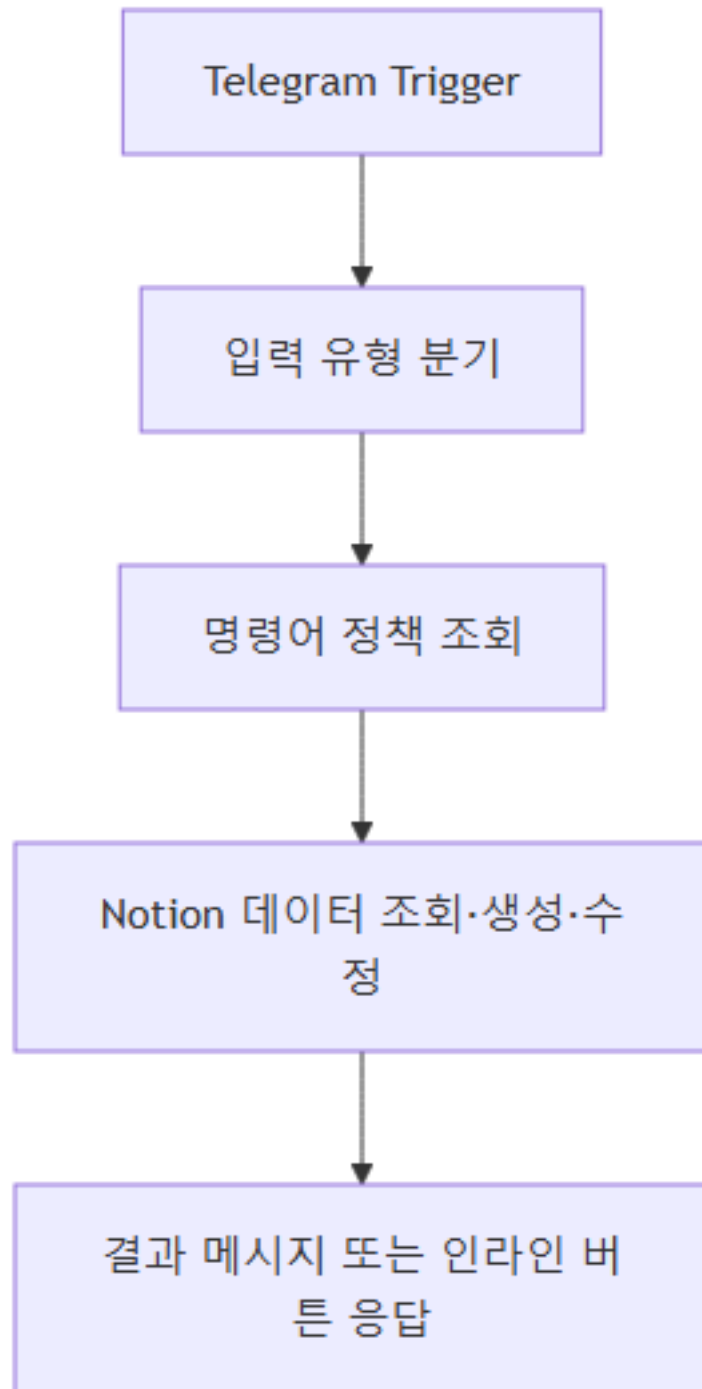
6. 플랫폼 제약 하의 유지보수 방식 정리

일부 대규모 수정 작업은 관리 API만으로 처리하기 어려운 구간이 있었고, 이 과정에서 저장 구조와 변경 이력을 이해한 뒤 안전하게 보정하는 방식으로 운영 절차를 정리했다. 중요한 점은 단순 사용이 아니라, 도구의 제약을 분석하고 운영 가능한 형태로 개선했다는 점이다.

대표 워크플로우

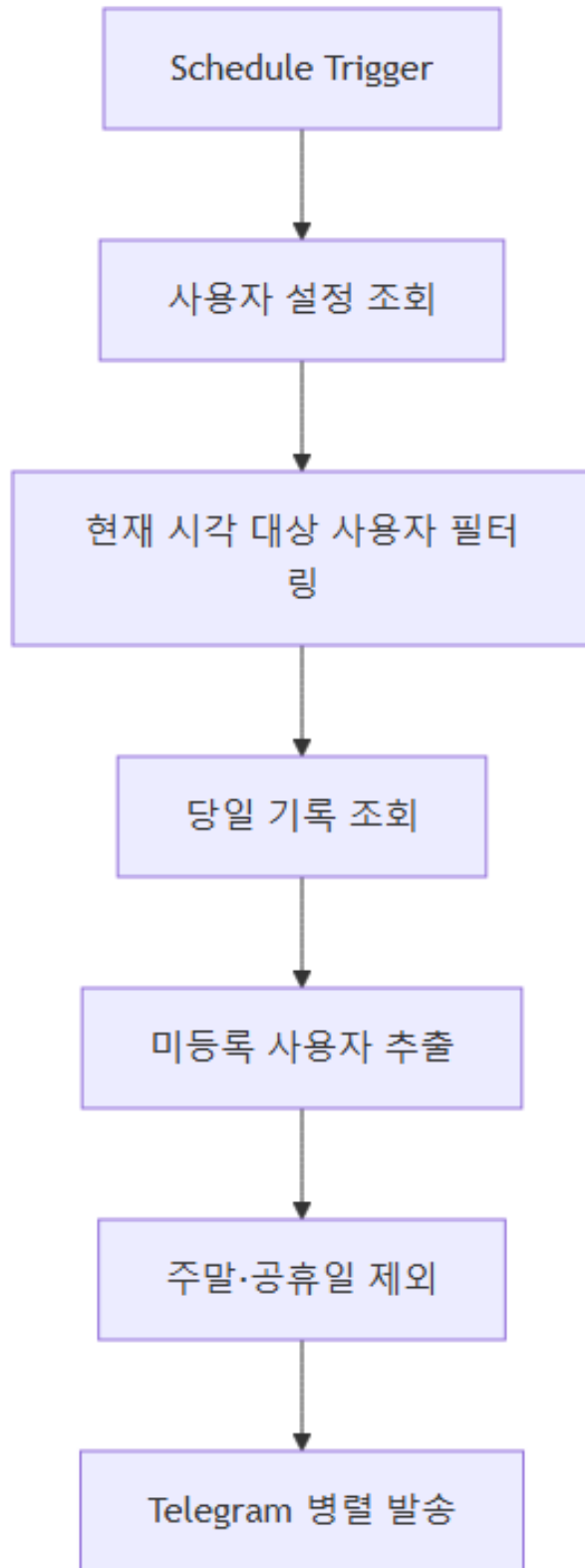
근태관리 워크플로우

핵심 워크플로우로, 텔레그램 메시지와 버튼 입력을 받아 근태를 기록·조회·수정한다.



출근 미등록 알림 워크플로우

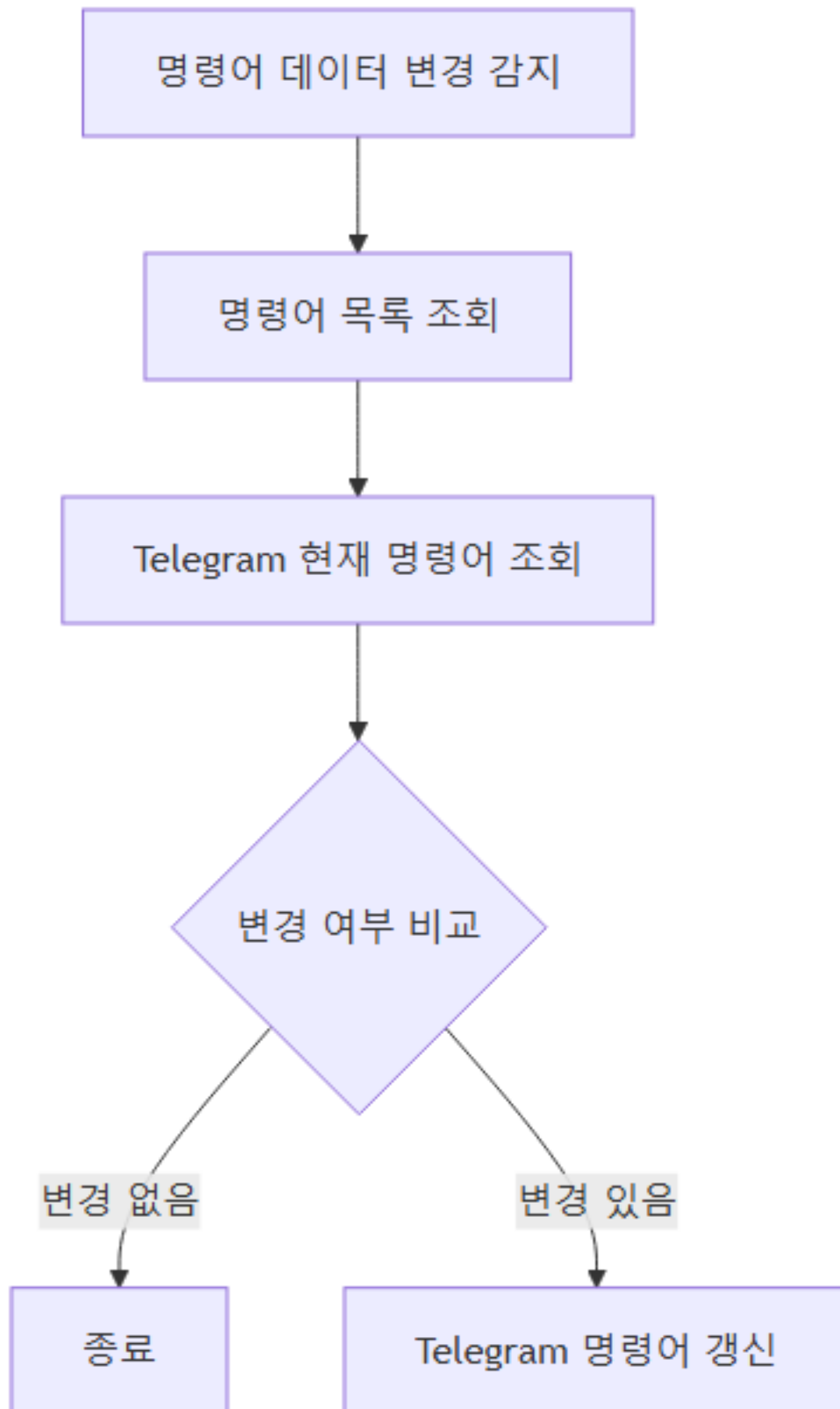
스케줄 기반으로 실행되며, 해당 시각에 출근해야 하는 사용자를 식별한 뒤 아직 기록이 없는 경우에만 알림을 발송한다.



- 사용자별 출근 기준 시각 반영
- 당일 기록 존재 여부 기반 알림 제외
- 공휴일 및 주말 예외 처리
- 병렬 발송을 통한 처리 시간 단축

명령어 동기화 워크플로우

명령어 데이터가 변경되면 Telegram 봇 명령 목록도 자동으로 동기화되도록 구성했다.



이를 통해 운영자는 코드를 수정하지 않고도 봇 메뉴를 최신 상태로 유지할 수 있다.

데이터 설계 요약

세부 식별자나 내부 스키마보다, 운영 구조가 명확하게 분리되어 있다는 점이 핵심이다.

데이터 영역	역할
근태 기록	출근, 퇴근, 외근, 복귀 등 이벤트 저장
사용자 설정	알림 기준 시각, 기본 프로젝트, 사용자별 옵션 관리
명령어 관리	봇 명령어, 설명, 동작 정책 정의
프로젝트 정보	외근 및 업무 연계용 프로젝트 선택 정보 저장

이 구조 덕분에 자주 바뀌는 운영 규칙을 코드가 아니라 데이터에서 제어할 수 있었다.

설계상 강점

NoCode + Code의 균형

기본 흐름은 n8n 노드로 구성하되, 플랫폼 기본 기능만으로 부족한 부분은 JavaScript Code 노드로 보완했다.

덕분에 개발 속도와 유연성을 동시에 확보할 수 있었다.

운영 친화적 구조

명령어, 사용자 설정, 프로젝트 목록을 데이터로 분리해 운영 중 변경 비용을 낮췄다.

이는 자동화 시스템이 단발성 데모가 아니라 실제 운영 가능한 형태가 되도록 만드는 핵심 요소였다.

예외 상황을 고려한 UX

자동 선택 실패 시 수동 선택으로 전환하고, 빈 조회 결과나 예외 케이스를 명시적으로 처리해 사용자 경험이 끊기지 않도록 설계했다.

비용 효율적 운영

상용 SaaS 의존을 최소화하고, self-hosted 환경과 무료·저비용 서비스 조합으로 실용적인 운영 구성을 완성했다.

사용 기술

영역	기술
자동화 엔진	n8n
데이터 저장	PostgreSQL, Notion API
인터페이스	Telegram Bot API
네트워크 구성	Cloudflare Tunnel
구현 언어	JavaScript
운영 환경	Docker, Ubuntu

정리

이 프로젝트는 메신저 기반 업무 자동화를 실제 운영 가능한 수준으로 설계하고, 플랫폼 제약과 운영 이슈를 직접 해결해낸 사례다.

특히 n8n 같은 자동화 도구를 단순 연결 수준이 아니라, 데이터 설계, 운영 안정성, 예외 처리, 사용자 경험까지 고려한 시스템으로 확장했다는 점에서 의미가 있다.

🚀 프로젝트: AI 비서 구축 (OpenClaw & Gemini 3.1)

1. 프로젝트 개요 (Executive Summary)

- 배경:** 단순 챗봇을 넘어, 개인의 실제 업무 데이터(Notion) 및 메신저(Telegram)와 연동되어 '행동'할 수 있는 진정한 의미의 AI 비서 필요성 대두.
- 목표:**
 - a. 오픈소스 MCP(Model Context Protocol) 게이트웨이인 OpenClaw의 안정적 인프라 구축.
 - b. 2026년 최신 모델인 Gemini 3.1 Flash Lite의 강력한 컨텍스트 처리 능력 활용.
 - c. 텔레그램을 통한 접근성 및 노션(Notion) 기반의 자동 보고/기록 시스템 완성.
- 성과:** AI 기반의 자동 업무 보고 체계를 구축하여 일일 보고서 작성 시간을 단축하고, 사내 AI 컨설팅을 위한 핵심 기술 실증(PoC) 완료.

2. 핵심 기능 (Key Features)

Gemini 3.1 기반의 지능형 대화

- 최신 모델인 gemini-3.1-flash-lite-preview를 탑재하여 압도적인 응답 속도와 정확도 확보.
- 긴 대화 맥락(Context Window)을 유지하며 복잡한 업무 지시 수행 가능.

텔레그램-노션 자동 보고 시스템 (Notion Integration)

- "쓰지 않아도 써지는 보고서": 텔레그램을 통한 말 한마디로 노션 데이터베이스에 업무 로그, 태스크, 일정을 규칙화하여 자동 기록.
- 파편화된 대화 내용을 AI가 분석하여 정제된 경영 지표 데이터로 자산화.

Docker 기반의 안정적인 온프레미스 구축

- 개인 Ubuntu 서버에 Docker Container 환경으로 구축하여 Host 시스템과 격리된 안전한 운영.
- 호스트 네트워크 모드 활용으로 향후 사내 인프라 점검 도구(커스텀 MCP) 확장을 위한 토대 마련.

3. 기술 스택 및 아키텍처 (Technical Stack)

구분	기술 요소	역할
Infrastructure	Ubuntu 24.04 LTS (Headless)	호스트 OS (15Gi RAM 환경)
Container	Docker & Docker Compose	애플리케이션 가상화 및 배포 관리
AI Gateway	OpenClaw (MCP Gateway)	LLM과 외부 인프라(메신저, DB) 연결
LLM	Google Gemini 3.1 Flash Lite	메인 AI 두뇌 (대화 및 로직 처리)
Interface	Telegram Bot API	사용자 인터페이스 (24/7 채널)
Database/Memory	Notion API	업무 로그 저장 및 장기 기억 장치

System Architecture Diagram

